

Verifier-Grounded Generate-Verify-Repair for Intent→Configuration NetOps Tasks: A Paired Mininet Emulation Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We preregistered and executed a provenance-traced paired confirmatory study (N = 200 intents; 400 paired episodes) to evaluate whether a deterministic verifier-grounded generate-verify-repair (GVR) loop reduces operationally detectable policy-violation errors when a hosted LLM produces device configuration sequences from operator intent in a Mininet emulation. Execution used shared_initial_candidate semantics (one sampled initial generation per intent reused by both baseline and guarded). Baseline success (no detected policy violation) = 196/200 (98%); guarded success = 200/200 (100%). Paired risk difference (guarded - baseline) = 0.02; preregistered exact McNemar two-sided p = 0.125 (primary confirmatory test). An exploratory paired bootstrap (seed = 1729, 10,000 resamples) yields a 95% percentile CI = [0.005, 0.04]. Four verifier-triggered repairs occurred (total hosted model calls = 204), giving mean extra model calls per intent = 0.02 and mean extra calls per avoided violation = 1.0. All reported numeric values, provider-call accounting, validator traces, and analysis artifacts are drawn verbatim from the archived execution and analysis bundle. We interpret the preregistered confirmatory result conservatively and discuss construct, internal, and external validity limits and operational implications.

I. INTRODUCTION

Automating intent→configuration translation is an active NetOps research direction because natural-language or intent descriptions are a natural operator interface but can be transformed incorrectly by large language models (LLMs). Mechanically checkable faults introduced by LLMs—syntactic errors, topology/reachability violations, and ACL/policy mistakes—can lead to availability loss or exposure if applied to devices without validation [1], [4]. Generate-Verify-Repair (GVR) patterns pair an LLM generator with a deterministic verifier: the verifier checks mechanizable constraints and, if violations occur, issues localized diagnostics used to request corrective edits from the model [2], [3]. Prototype work across code and domain-specific program generation suggests verifier-grounded repair reduces mechanically verifiable errors; however, questions remain about scaling such pipelines to heterogeneous multi-vendor template families, the concrete hosted-LLM cost per avoided violation, and how to interpret paired comparisons when sampling semantics reuse the initial model candidate.

Research question and contribution. Our preregistered research question is: Does a verifier-grounded generate-verify-repair loop significantly reduce operationally detectable policy-violation errors produced by a hosted LLM when

generating network configurations for operator intents across heterogeneous multi-vendor template families and topology patterns in a Mininet-based emulation? Contributions: (1) a provenance-traced confirmatory paired experiment (C1-GVR-multivendor-2026-v1) with shared_initial_candidate semantics executed on N = 200 intents (400 episodes); (2) audited provider and verifier accounting showing repair costs and concrete hosted-model calls; (3) artifact-grounded statistical inference (exact McNemar test) and exploratory bootstrap interval (seed = 1729, 10,000 resamples); (4) a focused analysis of failure modes, threats to validity, and operationally grounded recommendations consistent with archived artifacts.

II. RELATED WORK

Surveys and NetOps-specific reviews document rapid growth in LLM-for-NetOps work and recurring evaluation gaps—benchmark provenance, verifier integration, and limited multi-vendor emulation evidence [5], [6]. Prior prototypes show LLMs can draft device configurations but often introduce syntax, topology, or policy errors; verifier-guided repair appears as a recurrent mitigation pattern in code and domain program generation [2], [3]. NetConfEval and related benchmarks demonstrate feasibility of evaluating LLMs on configuration tasks but commonly lack provenance-traced verifier pipelines or multi-vendor emulation at scale [4]. Our study builds on these findings by providing a provenance-traced paired design across synthetic vendor templates in Mininet and by executing the study under explicit shared_initial_candidate semantics to isolate verifier/repair effects.

III. METHODOLOGY

Preregistration and estimand. The experiment was preregistered as C1-GVR-multivendor-2026-v1 before any model calls (preregistration SHA-256: 9e0fc992665835975b01289f7841926ef6abf3584252c5d0459ba0ecf166bb50). Primary estimand: paired_success_rate_difference_guarded_minus_baseline (probability that an intent yields zero operationally detectable violations under guarded minus baseline), tested with an exact McNemar two-sided test at $\alpha = 0.05$. Predeclared exploratory estimators include a paired bootstrap percentile CI (bootstrap_seed = 1729; resamples = 10,000) and descriptive cost metrics (mean extra hosted model calls per intent and mean extra calls per avoided violation).

Sampling, strata, and shared_initial_candidate semantics. Task selection was deterministic and stratified across three synthetic vendor-template families and four topology patterns (12 strata) to exercise template heterogeneity. $N = 200$ unique operator intents were fixed prior to model calls. Execution used shared_initial_candidate semantics: exactly one initial sampled generation per intent was produced and reused by both baseline and guarded episodes. Under these semantics baseline outcome is the deterministic validator verdict on that shared initial candidate; the guarded outcome equals the final guarded pipeline result after deterministic validation and any permitted repair. The preregistration explicitly declared that independent constrained-prompt arms were not executed; therefore paired differences can arise only from deterministic validator or repair steps, not from different first-pass samples or prompt templates.

Model configuration, sampling notation, and budget. Declared model (as archived) is gpt-5-mini (provider recorded as openai_responses). The archived model_configuration.json records temperature = null/unset; we explicitly state that null/unset is not evidence of deterministic provider sampling and does not imply temperature = 0. Planned budget enforced in the adapter: planned_episode_count = 400, initial_generation_calls_per_task = 1, maximum_repair_calls_per_task = 1, maximum_model_calls = 400.

Deterministic verifier and repair policy. deterministic_netops_validator_v5 is a rule-based verifier combining (i) syntactic parsing and command pattern checks, (ii) required-command sequencing and workflow policy checks (per-task payload), (iii) reachability/topology checks against the emulated Mininet state, and (iv) encoded ACL/policy checks derived from the task payload. The scorer rule full_intent_constraint_validation yields a binary success = "no detected violations" if the final configuration satisfies all mechanizable constraints. Guarded pipeline behavior: if the validator flags violations, a single verifier-triggered repair call may be issued; repair prompts contain the verifier's localized diagnostics requesting corrective edits. Repair calls were permitted only when the validator reported violations, and up to one repair call per intent was allowed per the preregistration.

Scoring, missingness, and analysis plan. Primary analysis: exact McNemar two-sided test on paired binary success indicators. Missingness and exclusions were predeclared: duplicate tasks (detected via deterministic hashing) or unrecoverable infra failures would be excluded and reported; ambiguous validator outputs were conservatively treated as violations in the primary analysis. Exploratory analyses included the paired bootstrap percentile CI (seed = 1729, 10,000 resamples) and descriptive cost accounting. Analysis code and seeds are archived.

IV. RESULTS

Execution completeness and timing. The preregistered run executed to completion without excluded pairs or unrecoverable infrastructure failures. The execution manifest

records start and completion times (started_at_utc = 2026-09-01T12:52:23.426943+00:00; completed_at_utc = 2026-09-01T13:19:15.308203+00:00) and shows planned_episode_count = 400 with completed_episode_count = 400 and failed_episode_count = 0. Deterministic reconciliation confirms complete_pair_count = 200 and that all deterministic consistency checks passed.

Audited provider accounting and stage decomposition. Provider auditing in the execution manifest reports hosted_model_call_event_count = 204 and decomposes these calls into stage_counts = {shared_initial_generation: 200, repair: 4}. The manifest further records cache_hit_count = 0 and cache_miss_count = 204. These audited counts are the authoritative provider-call accounting for the run and were used directly in cost summaries and downstream arithmetic.

Per-condition tallies and raw contingency. The archived condition summary (analysis/condition_summary.csv) reports: baseline successes = 196, baseline failures = 4 (observed baseline success rate = $196/200 = 0.98$); guarded successes = 200, guarded failures = 0 (guarded success rate = $200/200 = 1.0$). The paired 2×2 contingency (analysis/paired_contingency_table.csv) is reproduced exactly from the archived analysis artifact: n11 = 196 (both succeed), n10 = 4 (guarded succeed, baseline fail), n01 = 0 (guarded fail, baseline succeed), n00 = 0 (both fail). These contingency counts are the direct inputs to the preregistered confirmatory test and to exploratory interval estimation.

Confirmatory inference (preregistered). Using the preregistered exact McNemar two-sided test on the paired binary success indicators yields $p = 0.125$; under the preregistered $\alpha = 0.05$ threshold this result does not reject the null of no difference. The point estimate for the paired risk difference (guarded - baseline) computed from the contingency counts is 0.02 (2 percentage points). We report the exact test p-value and the risk difference as primary, preregistered outputs and treat any auxiliary intervals as exploratory per the preregistration multiplicity plan.

Exploratory bootstrap interval and caution. An exploratory paired bootstrap percentile interval was computed using the archived analysis procedure (bootstrap_seed = 1729; bootstrap_resamples = 10,000) and produced a 95% percentile CI for the paired risk difference = [0.005, 0.04]. We report this interval as supplementary magnitude information only and note the known instability of bootstrap percentile intervals when discordant pair counts ($n10 + n01$) are small; with only four discordant pairs the bootstrap estimate provides limited additional inferential resolution and should not supplant the exact McNemar inference for confirmatory claims. The analysis log (analysis/analysis_log.jsonl) contains the bootstrap seed and resample count used to produce this interval.

Repair events, cost accounting, and arithmetic derivations. Four verifier-triggered repair calls were invoked across the $N = 200$ intents, consistent with stage_counts.repair = 4 in the provider audit. Total hosted model calls therefore equal 200 shared initial calls + 4 repair calls = 204 audited calls. From these audited counts we compute mean hosted model

calls per intent = $204 / 200 = 1.02$ and mean extra calls per intent attributable to repairs = 0.02. The archived contingency shows that each repair converted a failing baseline outcome into a passing guarded outcome (the four n10 discordant pairs), so mean extra calls per avoided violation = 1.0 (4 repair calls / 4 avoided violations). All these arithmetic results are direct derivations from the archived provider and contingency artifacts and are reported verbatim.

Representative validator trace behavior and substantiation. The archived validator traces and scoring artifacts show the operational pattern that produced the contingency entries: for each pair, the `shared_initial_candidate` was passed to the deterministic verifier; when the verifier found mechanizable violations it produced localized diagnostics and (for four intents) the guarded pipeline issued a single repair call using those diagnostics. For those discordant pairs the pre-repair validator output recorded `violation_count > 0` and the repair decision was accompanied by a subsequent validator `validation_after` that recorded `violation_count = 0` and a final scorer decision of `success = 1`. Representative examples of valid `shared_initial_candidate` and their validator traces are provided in the archive (execution/scoring and execution/responses artifacts) and illustrate the exact `shared_initial_candidate` reuse semantics: the same initial generated candidate was scored as baseline and was the starting point for any guarded repair sequence. The reconciliation log confirms `cross_condition_cache_key_reuse_observed = false`; therefore the guarded conversions arose from validator/repair activity rather than from independent new initial samples.

Power context and interpretation. The preregistered power calculation targeted detecting a 10 percentage point absolute reduction with approximate required $N \approx 157$; $N = 200$ was chosen to provide margin and permit stratified description. The observed baseline failure prevalence ($4/200 = 2\%$) is substantially lower than the planning scenario used for power targeting, which reduces power to detect small absolute improvements and yields a small discordant count (4) that underlies the non-significant McNemar result. We therefore present the exact McNemar $p = 0.125$ as the formal confirmatory outcome, report the exploratory bootstrap interval (seeded and archived), and interpret the cost accounting and repair effectiveness as artifact-grounded operational diagnostics rather than as definitive evidence of broad efficacy beyond the constrained emulation and verifier scope.

V. DISCUSSION

Causal interpretation under `shared_initial_candidate` semantics. Because execution used `shared_initial_candidate` semantics (one sampled initial generation per intent reused by baseline and guarded), paired improvements arise solely from deterministic validator/repair actions. The preregistration explicitly declared that independent constrained-prompt arms were not executed; thus the observed $n10 = 4$ discordant pairs reflect validator-triggered repairs that converted failing shared initial candidates into passing final configurations. Any

statements attributing improvements to prompt-engineering or differing first-pass samples would be incorrect for this run.

Construct validity and verifier coverage. The deterministic `netops_validator_v5` covers syntactic correctness, required command sequencing, reachability/topology checks against the emulated state, and encoded ACL/policy checks derived from per-task payload constraints. This verifier can only detect violations expressible in its rules; higher-order semantic misinterpretations not captured by those encodings remain undetectable and are identified in our limitations. The archived `validator_trace` objects for the four discordant pairs show the same pattern: (1) `validation_before` included explicit violation codes and localized diagnostic messages produced by `deterministic_netops_validator_v5`; (2) a single repair call was initiated with the verifier diagnostics embedded in the repair prompt; (3) `validation_after` recorded `repair_applied = true` and `violation_count = 0`. These per-pair trace sequences directly substantiate the causal pathway from verifier diagnostics $\rightarrow$ repair call $\rightarrow$ corrected final configuration.

Provider audit and stage accounting implications. The execution manifest and deterministic_reconciliation report hosted `model_call_event_count = 204`, decomposed into 200 `shared_initial_generation` events and 4 repair calls, with `cache_hit_count = 0` and `cache_miss_count = 204`. Stage_counts show `shared_initial_generation: 200` and `repair: 4`; condition_counts show `shared: 200` and `guarded: 4`. These audited counts confirm that (a) exactly one initial sampled candidate was produced per intent and reused across both conditions, and (b) repairs were rare and invoked only for the four discordant intents. The arithmetic in Results—mean hosted calls per intent = 1.02 and mean extra calls per avoided violation = 1.0—follows directly from these manifest numbers and the contingency table; these cost metrics should be interpreted as emulation-setting observations tied to the specific verifier and repair policy.

Statistical interpretation with sparse discordance. The primary confirmatory procedure (exact McNemar two-sided test) produced $p = 0.125$. Small discordant counts ($n10 + n01 = 4$) reduce the power of the exact paired test to detect small absolute differences; the preregistered power calculation assumed a larger baseline failure prevalence and a 10 percentage-point target effect. The exploratory bootstrap percentile CI (seed = 1729; 10,000 resamples) provides a complementary magnitude estimate (0.005–0.04) but must be treated cautiously in light of sparse discordance: bootstrap percentile intervals rely on resampling variability that is limited when the number of discordant pairs is small, which can yield intervals that are sensitive to resampling variability. We therefore present the bootstrap CI as exploratory magnitude information and retain the exact McNemar result as the preregistered confirmatory inference.

Failure-mode diagnostics and verifier blind spots. The four baseline failures corrected by repairs were all mechanics-expressible failures that `deterministic_netops_validator_v5` could localize (e.g., missing required sequence steps, interface-state precondition

violations). Across the 200 intents, validator traces record categories of detected violations (syntax, missing required commands, sequence/order violations, and reachability constraint failures). However, the verifier does not represent certain semantic correctness classes—misinterpretations of informal intent that nevertheless produce syntactically well-formed and reachability-consistent configurations (for example, incorrect ACL semantics not expressible in the task payload). These false-negative classes are an important residual risk and explain why deterministic verification is necessary but not sufficient for full semantic correctness.

Design implications for GVR deployments. Artifact-grounded evidence from this run suggests concrete operational design tradeoffs: when a deterministic verifier can (i) express the operational constraints of interest and (ii) produce localized diagnostics that can be encoded into corrective prompts, a bounded repair budget (here one repair per failing intent) can eliminate mechanically detectable faults at low hosted-model overhead in emulated template-based settings. Quantitatively, in our emulation the repair cost per avoided violation was 1.0 extra model call on average, and repairs occurred for only 2% of intents. These numbers are conditional on the verifier’s detection power, the templates’ failure modes, and the `shared_initial_candidate` design. Practitioners should therefore view GVR budget planning as verifier-conditioned: stronger verifier coverage may both detect more failures (raising repair invocation counts) and prevent silent semantic errors (reducing residual risk).

Recommendations grounded in archived artifacts. Based on the archived manifest and validator traces we offer practical, artifact-grounded recommendations: (1) extend deterministic verifier expressivity to codify more semantic/policy ontologies so fewer semantic errors remain undetected; (2) when the goal is to separate sampling/prompt-engineering effects from repair effects, preregister and execute independent multi-sample initial generations per condition (the `shared_initial_candidate` semantics used here intentionally isolates repair effects only); (3) collect and report provider audit counts and per-pair validator traces to make cost/benefit tradeoffs transparent to operators; and (4) design follow-on confirmatory runs to target higher baseline failure prevalence or larger N when the expected paired effect is small, since power depends on baseline failure rate and discordant counts. Each recommendation above is supported by the archived `execution_manifest`, `deterministic_reconciliation`, and `validator_trace` artifacts.

Limitations reiterated with technical specificity. The conclusions above are bounded by concrete technical constraints recorded in the archive: single declared hosted model (gpt-5-mini), synthetic vendor templates and Mininet emulation, a deterministic verifier with limited semantic coverage, and `shared_initial_candidate` semantics that preclude attributing effects to altered first-pass sampling or prompt templates. These limitations are documented in the execution and analysis artifacts and motivate the targeted next steps described above.

VI. LIMITATIONS

- Scope limited to Mininet-style emulation with synthetic, provenance-traced intent→configuration tasks; results do not generalize directly to production hardware or live operations.
- Single declared model (gpt-5-mini) evaluated; multi-model generality is not established.
- Deterministic validator coverage limited to mechanically checkable errors (syntax, reachability, ACL/policy encodings); higher-level semantic or specification misinterpretations outside the validator may remain undetected.
- `Shared_initial_candidate` semantics imply observed paired differences derive only from verifier-triggered repair; this design does not measure effects of alternative prompting strategies or independent multi-sample candidate selection.
- Observed confirmatory effect (2 percentage points) did not reach statistical significance at $\alpha = 0.05$ for $N = 200$ (exact McNemar $p = 0.125$); low baseline failure prevalence (2%) reduced power to detect small absolute improvements under some preregistered scenarios.

VII. CONCLUSION

We preregistered and executed a provenance-traced paired confirmatory study (C1-GVR-multivendor-2026-v1) using `shared_initial_candidate` semantics to measure whether a deterministic verifier plus bounded repair reduces mechanically detectable policy violations for intent→configuration tasks in a Mininet emulation. The preregistered exact McNemar two-sided test did not reject at $\alpha = 0.05$ ($p = 0.125$). Guarded GVR invoked four repairs and corrected the four observed baseline violations at low model-call overhead (model calls = 204; mean extra calls per intent = 0.02), yielding a paired risk difference estimate of 0.02 and an exploratory paired bootstrap CI = [0.005, 0.04] (bootstrap_seed = 1729, resamples = 10,000). These artifact-grounded results indicate that when a deterministic verifier can express and check relevant constraints and provide localized feedback, a small repair budget can eliminate mechanically detectable policy violations in similar template-based emulated settings. The results do not establish production readiness or multi-model generality. All reported numbers, provider accounting, validator traces, and analysis artifacts are drawn verbatim from the archived execution and analysis bundles referenced in the preregistration and execution manifests.

References [1] R. Mondal, A. Tang, R. Beckett, T. Millstein, and G. Varghese, "What do LLMs need to Synthesize Correct Router Configurations?", *Proceedings of the ACM*, 2023, doi: 10.1145/3626111.3628194. [2] R. Cadenas, "Convergent Correctness in Stochastic Code Generation: A Generate-Verify-Repair Architecture for Deterministic Validation of Autonomous AI Coding Agents in Regulated Environments", *SSRN*, 2026, doi: 10.2139/ssrn.6754899. [3] Z. Ding, "Executable but Wrong: Verifier-Grounded Contracts and Counterexamples for LLM-Generated Music Programs", 2026, doi: 10.31224/7995. [4] C. Wang, M. Scazzariello, A.

Farshin, S. Ferlin, D. Kostić, and M. Chiesa, "NetConfEval: Can LLMs Facilitate Network Configuration?", Proceedings of the ACM, 2024, doi: 10.1145/3656296. [5] S. Y. Long et al., "A Survey on Intelligent Network Operations and Performance Optimization Based on Large Language Models", IEEE Communications Surveys & Tutorials, 2025, doi: 10.1109/comst.2025.3526606. [6] L. Zhang et al., "A Survey of AIOps in the Era of Large Language Models", Proceedings of the ACM, 2025, doi: 10.1145/3746635.

DISCLOSURE STATEMENT

Declared model (archived): gpt-5-mini (provider recorded as openai_responses). Adapter family: hosted_netops_gvr_v1. Deterministic validator: deterministic_netops_validator_v5. Task generator: deterministic_netops_task_generator_v5. Preregistration: C1-GVR-multivendor-2026-v1 (SHA-256 = 9e0fc992665835975b01289f7841926ef6abf3584252c5d0459ba0ecf166bb50), recorded prior to any model calls. Initial master prompt archived at provenance/master_prompt.txt (SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba). Human role: a human Research Director selected the topic family and constrained the initial master prompt prior to the autonomy lock; after the lock the AI autonomously performed literature verification, preregistration, experiment execution, analysis, peer review, revision, and manuscript drafting; no human manually edited technical manuscript content after the autonomy lock. Sampling semantics: execution used shared_initial_candidate (exactly one sampled initial generation per intent was produced and reused by both baseline and guarded); archived model configuration records temperature = null/unset (null/unset is not evidence of deterministic provider sampling and does not imply temperature = 0). Provider accounting (audited): hosted_model_call_event_count = 204 decomposed as 200 shared initial generations + 4 repair calls. Reported numbers, provider-call accounting, validator traces, and analysis artifacts are drawn verbatim from the archived execution and analysis bundles referenced in the preregistration and execution manifests.

REFERENCES

- [1] Rajdeep Mondal, Alan Tang, Ryan Beckett, Todd Millstein, George Varghese, "What do LLMs need to Synthesize Correct Router Configurations?", 2023, 10.1145/3626111.3628194
- [2] Rafael Cadenas, "Convergent Correctness in Stochastic Code Generation: A Generate-Verify-Repair Architecture for Deterministic Validation of Autonomous AI Coding Agents in Regulated Environments", 2026, 10.2139/ssrn.6754899
- [3] Zhengyang Ding, "Executable but Wrong: Verifier-Grounded Contracts and Counterexamples for LLM-Generated Music Programs", 2026, 10.31224/7995
- [4] Changjie Wang, Mariano Scazzariello, Alireza Farshin, Simone Ferlin, Dejan Kostić, Marco Chiesa, "NetConfEval: Can LLMs Facilitate Network Configuration?", 2024, 10.1145/3656296
- [5] S.Y. Long, Jingjing Tan, Bomin Mao, Fengxiao Tang, Yangfan Li, Ming Zhao, Nei Kato, "A Survey on Intelligent Network Operations and Performance Optimization Based on Large Language Models", 2025, 10.1109/comst.2025.3526606
- [6] Lingzhe Zhang, Tong Jia, Mengxi Jia, Yifan Wu, Aiwei Liu, Yong Yang, Zhonghai Wu, Xuming Hu, Philip S. Yu, Ying Li, "A Survey of AIOps in the Era of Large Language Models", 2025, 10.1145/3746635